Reading: Agentic AI Protocols

Estimated time needed: 10 minutes

What are AI agent protocols?

AI agent protocols establish standards of communication among artificial intelligence agents and between AI agents and other systems. These protocols specify the syntax, structure and sequence of messages, along with communication conventions such as the roles agents take in conversations and when and how they respond to messages.

Agent-based AI systems often run in silos. They're built by different providers using diverse AI agent frameworks and employing distinct agentic architectures. Real-world integration becomes a challenge, and coupling these fragmented systems requires tailored connectors for all possible types of agent interaction.

This is where protocols come in. They turn disparate multi-agent systems into an interlinked ecosystem where AI-powered agents share a way of discovering, understanding and collaborating with each other.

Although agentic protocols are part of AI agent orchestration, they don't act as orchestrators. They standardize communication but don't manage agentic workflow coordination, execution and optimization.

Examples of AI agent protocols

Many protocols are still in their early stages, so they have yet to be widely used or applied on a large scale. This lack of maturity means that organizations must be prepared to act as early adopters, adjusting to breaking changes and evolving specifications.

As agentic technology evolves, new protocols might emerge. Here are some current common AI agent protocols:

- Agent Communication Protocol (ACP)
- Agent Network Protocol (ANP)
- Agent-User Interaction (AG-UI) Protocol
- Agent2Agent (A2A) Protocol
- Model Context Protocol (MCP)
- Agent Payments Protocol (AP2)

Agent Communication Protocol (ACP)

ACP is an open standard for agent-to-agent communication. With this protocol, we can transform our current landscape of siloed agents into interoperable agentic systems with easier integration and collaboration.

With ACP, originally introduced by IBM's BeeAI, AI agents can collaborate freely across teams, frameworks, technologies and organizations. It's a universal protocol that transforms the fragmented landscape of today's AI agents into interconnected teammates and this open standard unlocks new levels of interoperability, reuse and scale. As the next step following the Model Context Protocol (MCP), an open standard for tool and data access, ACP defines how agents operate and communicate.

For more information, visit https://www.ibm.com/think/topics/agent-communication-protocol#1190488334.

Note: In case you are unable to access the link, right-click and open it in a new tab.

Agent Network Protocol (ANP)

The Agent Network Protocol (ANP) is an open-source communication framework created specifically for intelligent agents. It functions much like HTTP but is tailored for a future where agents are the primary entities operating online. ANP allows these agents to locate, connect with, and interact across the internet, fostering an open and secure environment for collaboration between agents.

ANP addresses a core challenge in the AI ecosystem: the lack of a standardized, secure, and efficient method for agent-to-agent communication. By offering a unified protocol, ANP lays the groundwork for seamless interaction among agents in the evolving landscape of the AI-driven internet.

For more information, visit https://www.agent-network-protocol.com/guide/.

Agent-User Interaction (AG-UI) Protocol

AG-UI is a streamlined, open protocol based on events, designed to standardize the connection between AI agents and user-facing applications.

It emphasizes ease of use and adaptability, providing a consistent structure for the exchange of agent state, user interface intents, and user interactions between your agent and front-end applications. This enables developers to quickly build and deploy agent-driven features that are stable, easy to debug, and user-friendly—without needing to rely on complex, custom integrations, allowing them to focus on core application functionality.

For more information, visit: https://docs.ag-ui.com/introduction.

Agent2Agent (A2A) Protocol

The A2A protocol is an open standard for AI agent communication initially launched by Google and now managed under the Linux Foundation. It follows a client-server model setup with a three-step workflow:

1. Discovery occurs when an entity (a human user or another AI agent) initiates a task request to a client agent, which then looks up remote agents to determine the best fit.
2. Once the client agent identifies a remote agent capable of fulfilling the task, it then goes through authentication. The remote agent is responsible for authorization and granting access control permissions.
3. Communication proceeds with the client agent sending the task and the remote agent processing it. Agent-to-agent communication happens over HTTPS for secure transport, with JSON-RPC (Remote Procedure Call) 2.0 as the format for data exchange.

For more information, visit https://www.ibm.com/think/topics/agent2agent-protocol#1509394338.

Note: In case the link doesn't open, please right-click and open the link in a new tab.

Model Context Protocol (MCP)

Introduced by Anthropic, the Model Context Protocol (or MCP) provides a standardized way for AI models to get the context they need to carry out tasks. In the agentic realm, MCP acts as a tier for AI agents to connect and communicate with external services and tools, such as APIs, databases, files, web searches and other data sources.

MCP encompasses these three key architectural elements:

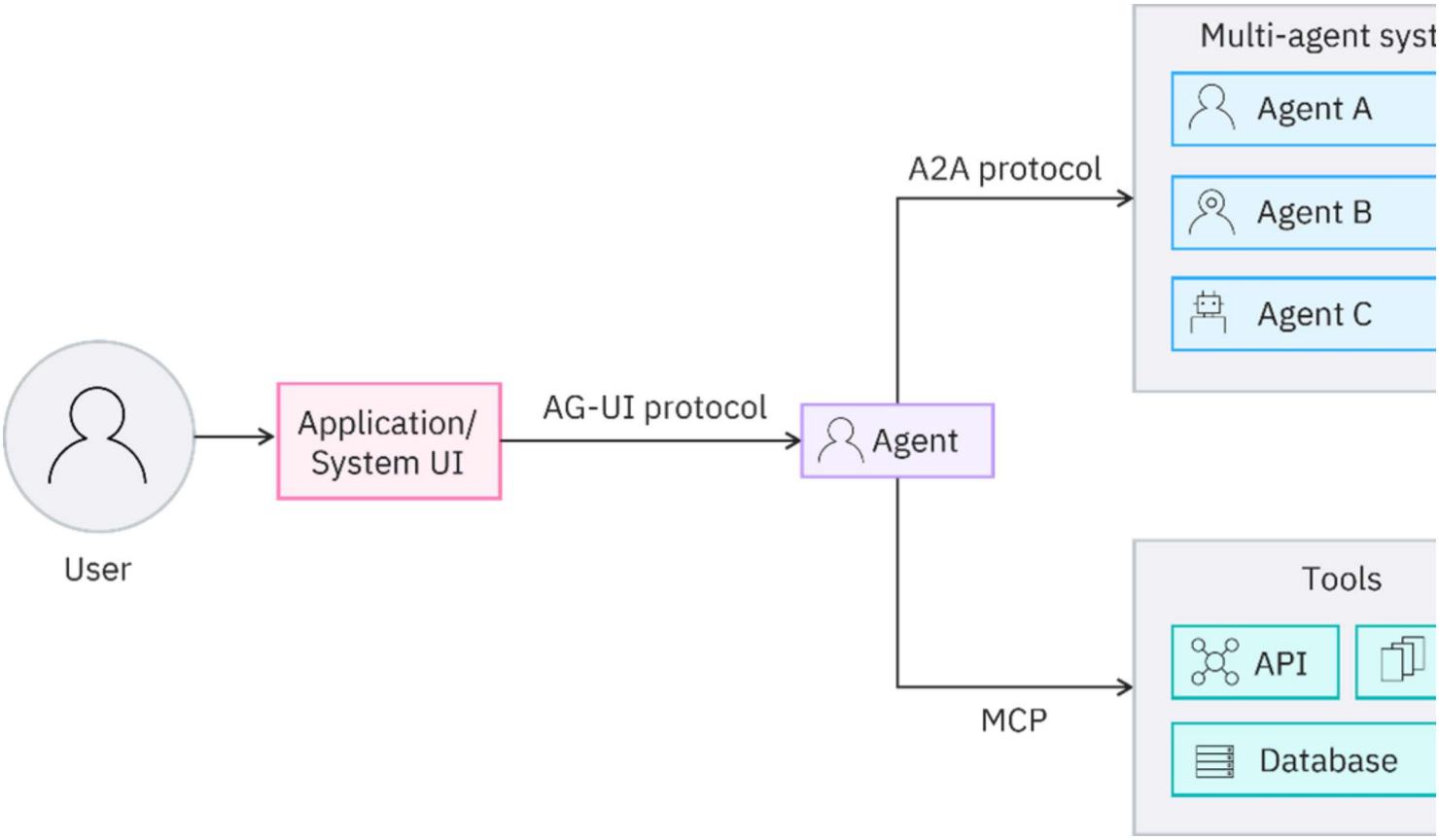
- The MCP host contains orchestration logic and can connect each MCP client to an MCP server. It can host multiple clients.
- An MCP client converts user requests into a structured format that the protocol can process. Each client has a one-to-one relationship with an MCP server. Clients manage sessions, parse and verify responses and handle errors.
- The MCP server converts user requests into server actions. Servers are typically GitHub repositories available in various programming languages and provide access to tools. They can also be used to connect LLM inferencing to the MCP SDK through AI platform providers such as IBM and OpenAI.

In the transport layer between clients and servers, messages are transmitted in JSON-RPC 2.0 format using either standard input/output (stdio) for lightweight, synchronous messaging or HTTP for remote requests.

For more information, visit https://www.ibm.com/think/topics/model-context-protocol#179371084.

Note: In case the link doesn't open, please right-click and open the link in a new tab.

The diagram below shows a simplified example of these last three protocols working together in a typical multi-agent environment.



Agent Payments Protocol (AP2)

In September 2025, Google unveiled a new standard called Agent Payments Protocol (or AP2). It is built in collaboration with leading players in the payments and technology sectors. AP2 is designed to enable secure, cross-platform transactions, initiated by AI agents. It can also function as an extension to existing protocols such as Agent2Agent (A2A) and the Model Context Protocol (MCP).

Unlike traditional payment systems that rely on a person clicking a 'Buy Now' button, AP2 is designed for scenarios where autonomous agents make purchases on behalf of users.

To support this, a system is needed that can:

- Reflect the user's intent
- Allow the agent to operate within defined boundaries
- Guarantee that transactions are traceable and secure

AP2 addresses this using a concept called 'Mandates', which are cryptographically signed digital instructions.

There are two types of mandates; **Intent Mandates** and **Cart Mandates**, and they support the two main ways users interact with agents while shopping.

User-involved purchases: These are real-time purchases made with a human present, such as requesting an agent to *"Buy me the cheapest available return ticket to Bali in October."*

That purchase request is recorded as an **Intent Mandate**. It captures the context of your request in a verifiable way, forming the foundation for the transaction. Once the agent shows you a cart with selected items, your confirmation creates a **Cart Mandate**. This step finalizes the transaction with a secure, tamper-proof record of exactly what you approved—items, price, and terms—ensuring transparency and trust.

Delegated purchases: These are purchases where the user is not present and involved at the time of the transaction itself. For example, your request might be to *"Buy me two of the best available theatre tickets as soon as they become available, up to a maximum spend of $250."*

In this scenario, you provide a pre-signed **Intent Mandate** ahead of time. This mandate outlines specific criteria such as spending caps, timings, and other restrictions. It acts as proof of your authorization and gives the agent permission to proceed. Once the mandate's criteria are met, the agent can autonomously generate and sign a **Cart Mandate** to complete the purchase on your behalf.

These two mandate types enable agents to make payments safely on behalf of users, while also providing merchants and payment providers with assurance that each transaction is properly authorized and trustworthy.

For more information, visit https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol, and see the YouTube video at https://www.youtube.com/watch?v=yLTp3ic2j5c.

Note: In case the link doesn't open, please right-click and open the link in a new tab.

How do the A2A, MCP, and AP2 protocols work together for agentic commercial transactions?

The A2A, MCP, and AP2 protocols work together to form the framework of agentic commercial transactions.

To understand how these three protocols work together in agentic commercial transactions, let's look at a simple example:

1. **User request:** "Order me a new pair of wireless headphones with noise cancellation, under $250."
2. **A2A at work:** The shopping agent communicates with a retailer's product agent and a payment service agent to manage the process.
3. **MCP at work:** The agent gathers product details, user preferences, and past purchase history via the Model Context Protocol.
4. **AP2 at work:** After selecting a suitable product, the agent creates a Cart Mandate. Once the user reviews and approves it, the payment agent securely processes the transaction.

Criteria for choosing an AI agent protocol

With the lack of benchmarks for standardized evaluation, enterprises must conduct their own appraisal of the protocol that best fits their business needs. They might need to start with a small and controlled use case combined with thorough and rigorous testing.

Here are a few aspects to keep in mind when assessing agent protocols:

- Efficiency
- Reliability
- Scalability
- Security

Efficiency

Ideally, protocols are designed to limit latency, resulting in swift data transfer and rapid response times. Although some communication overhead is expected, it must be kept to a minimum.

Reliability

AI agent protocols must be able to handle changing network conditions throughout agentic workflows, with mechanisms in place to manage failures or disruptions. For instance, ACP is designed with asynchronous communication as the default, which suits complex or long-running tasks. Meanwhile, A2A supports real-time streaming using SSE for large or long outputs or continuous status updates.

Scalability

Protocols must be robust enough to cater to growing agent ecosystems without compromising performance. Evaluating scalability can include increasing the number of agents or links to external tools over a period of time, either gradually or suddenly, to observe how a protocol operates in those conditions.

Security

Maintaining security is paramount, and agent protocols are increasingly incorporating safety guardrails. These include authentication, encryption and access control.

Summary

In this reading, you learned that:

- AI agent protocols establish standards for communication among AI agents and between agents and external systems, defining message structure, syntax, roles, and response conventions.
- Many AI agents operate in isolated silos with different frameworks, making real-world interoperability difficult without standardized protocols.
- Protocols enable discovery, understanding, and collaboration among agents, forming an interconnected ecosystem.
- Common agent communication protocols include ACP (Agent Communication Protocol), ANP (Agent Network Protocol), AG-UI (Agent-User Interaction Protocol), A2A (Agent2Agent Protocol), MCP (Model Context Protocol), and AP2 (Agent Payments Protocol).
- Commercial and transaction protocols such as A2A, MCP, and AP2 facilitate agent-based transactions.
- A2A protocol manages discovery, authentication, and secure task communication between agents; MCP provides standardized access to external tools, APIs, and data for agents; AP2 enables autonomous, agent-initiated, secure payments with digitally signed mandates.

- A typical integration workflow of a user request would be that A2A handles the communication, MCP gathers the context, and AP2 manages the secure payment, enabling seamless multi-protocol collaboration.
- Enterprises selecting protocols must assess efficiency, reliability, scalability, and security, and might need to pilot use cases due to evolving standards, lack of benchmarks, and ongoing technical improvements.